

AI 감정 재구성과 친구 응원을 결합한 자기 칭찬 기반 소셜 플랫폼



팀명: 칭찬고래 (분과: B)

최종보고서

202355544 서혜민 (gp4077@pusan.ac.kr)

202355529 김태란 (taeran4767@pusan.ac.kr)

202355640 오지현 (zeehy78@pusan.ac.kr)

지도교수: 조준수

제출일: 2026년 9월 14일

부산대학교

목차

1	서론	1
1.1	과제 배경	1
1.2	필요성	1
1.3	과제 목표 및 핵심 가치	2
1.4	최종 산출물 개요	2
2	요구사항 분석 및 설계 변경 이력	3
2.1	서비스 요구사항 구체화 및 디자인 협업	3
2.2	설계 변경 개요 (착수 → 중간 → 최종)	3
2.3	지도교수 피드백 및 반영 방향 - 최종 평가표 기준	4
2.4	기능적 요구사항 (최종)	4
2.5	비기능적 요구사항 (최종)	6
3	시스템 설계 및 아키텍처	7
3.1	전체 시스템 아키텍처	7
3.2	기술 스택	9
4	구현 상세	10
4.1	AI 칭찬 코칭 파이프라인	10
4.2	개인 기록 검색	10
4.3	출력 검증 및 자동 재시도	10
4.4	안전 계층	11
4.5	모바일 앱 및 서비스 백엔드 구현	11
4.6	인프라 및 배포	12
4.7	운영 성능 계측 및 개선	13
5	데이터 수집 및 성능 평가	15
5.1	학내 축제 부스 운영 및 실사용자 데이터 확보	15

5.1.1	데이터 정제 및 분류	15
5.1.2	합성 데이터 생성 파이프라인	16
5.2	AI 응답 품질 평가 프레임워크	16
5.2.1	이용 지표 수집	17
5.3	베타 테스트 설계	18
6	산학 자문 의견 반영	19
7	개발 일정 및 구성원별 역할	21
7.1	최종 개발 일정	21
7.2	구성원별 역할 및 수행 내용	23
8	결론 및 향후 계획	24
8.1	결론	24
8.2	향후 계획	24

제1장 서론

1.1 과제 배경

자기 자신에 대한 평가는 일상의 동기와 정서에 지속적으로 영향을 미친다. 특히 청년층에서는 성취에 대한 기준이 높아 스스로를 과도하게 비판하는 경향이 관찰되며, 이러한 자기비판적 사고는 우울감, 무기력, 낮은 자기효능감과 연결되는 것으로 알려져 있다.

본 과제의 출발점은 단순하다. **나에 대한 칭찬을 매일 기록하면, 나를 바라보는 시선이 긍정적으로 바뀐다.** “세 시간밖에 못 했다”는 자책이 “세 시간이나 해냈다”는 인정으로 전환될 때, 사용자는 자신의 하루를 다르게 해석하기 시작한다. 나아가 이 습관이 타인에게 확장되면, 내가 건넨 응원이 상대방에게 닿고 상대방의 응원이 다시 나에게 돌아오는 **칭찬의 선순환**이 형성된다.

1.2 필요성

이러한 문제의식의 유효성을 확인하기 위해 기록·공유 행동과 자기 칭찬에 대한 인식을 묻는 사전 설문조사를 수행하였다. 총 37건의 응답 중 응답자의 대부분(35명, 94.6%)이 자신의 성취를 공유하고 싶은 때가 있다고 답하여, 공유 기능에 대한 수요가 뚜렷하게 나타났다.

다만 그 공유의 범위는 제한적이었다. 함께 사용하고 싶은 대상으로 친한 친구를 선택한 응답이 27명(73.0%, 복수응답)이었던 반면, 모르는 사람의 칭찬이 불편하며 지인만 보기를 원한다는 응답이 12명(32.4%), 칭찬이 형식적으로 느껴질 것을 우려한 응답이 16명(43.2%, 복수응답)으로 나타났다. 주관식 응답에서도 자기표현이 자랑처럼 보이는 것에 대한 부담이 확인되었다.

사용자는 자신의 기록을 공유하고 싶어 하면서도, 그 대상이 불특정 다수이거나 칭찬이 형식적으로 오가는 상황은 부담으로 느낀다. 본 과제는 이를 **친구 중심의 공유 구조, 사용자가 선택하는 공개 범위, 원문 맥락에 근거한 AI 코칭**이라는 설계 방향으로 반영하였다.

1.3 과제 목표 및 핵심 가치

본 과제의 핵심 가치는 “스스로를 칭찬하는 습관”을 구조적으로 보존하는 데 있다. 이 가치는 이후 서술할 여러 설계 결정(특히 AI 역할의 재정의)의 기준점이 되었다. 구체적인 목표는 다음과 같다.

- 사용자가 짧은 시간 안에 자기 칭찬을 기록하고, 필요 시 AI의 코칭 한마디를 참고하여 스스로 글을 완성하도록 돕는다.
- 친구와 자기 칭찬을 공유하고 서로 응원하는 소셜 경험을 제공하여, 기록의 지속성과 관계적 만족을 함께 확보한다.
- 소형 언어 모델을 실서비스에 안정적으로 통합하고, 응답 품질을 정량적으로 검증할 수 있는 파이프라인과 평가 체계를 구축한다.

1.4 최종 산출물 개요

최종 산출물은 크게 네 가지로 구성된다. 첫째, React Native(Expo) 기반의 크로스플랫폼 모바일 애플리케이션이다. 둘째, Supabase 기반의 서비스 백엔드로 인증, 데이터 관리, 이미지 저장 및 푸시 알림을 담당한다. 셋째, FastAPI 기반의 AI 서버로 개인 기록 검색, 칭찬 코칭 생성, 출력 검증을 담당하며 AWS ECS 환경에서 운영한다. 넷째, 실사용자 데이터에 근거한 프롬프트·예시 풀과 AI 응답 품질 평가 프레임워크이다.

제2장 요구사항 분석 및 설계 변경 이력

2.1 서비스 요구사항 구체화 및 디자인 협업

사전 조사와 기획 내용을 구현 가능한 단위로 구체화하기 위해 기능명세서와 정보구조도를 작성하였다. 기능별 상세 동작, 입력·출력 및 예외 조건을 정리하고, 홈·쓰기·보관함·마이페이지를 중심으로 화면과 모달의 이동 관계를 정의하였다. 이를 디자인·개발 담당자가 공유하는 기준으로 활용하였다.

디자인-기획 회의에서는 무드보드를 통해 컨셉을 비교하고, 스타일 가이드와 쓰기·피드 중심의 핵심 화면, 부가 화면 순으로 작업 일정을 조율하였다. 마이크로인터랙션은 핵심 화면 구체화 이후 협의하도록 우선순위를 조정하였다.

온보딩에서 칭찬 스타일을 유형별로 파악하여 AI 응답에 반영하는 방안도 검토하였으나, 설문 문항의 학술적 근거와 타당성이 충분히 확보되지 않아 MVP 범위에서 제외하였다. 유형 기반 개인화는 별도의 검증 이후 재검토하도록 정리하였다.

2.2 설계 변경 개요 (착수 → 중간 → 최종)

착수보고 이후 지도교수 피드백과 초기 프로토타이핑 결과를 반영하여 AI 파이프라인의 범위와 구조를 전면 재정의하였다. 착수보고서에서 제시한 7단계 파이프라인은 기술적으로는 구현 가능하나, MVP 단계에서 사용자 문제 해결에 기여하는 비중 대비 복잡도가 과도하다고 판단하였다.

또한 부산대학교 축제 기간에 운영한 홍보 부스를 통해 실사용자로부터 자기 칭찬 원문과 응원 응답 총 300건을 직접 수집하였다. 이 데이터는 프롬프트의 말투 규칙, few-shot 예시 풀, 평가 앵커를 실제 사용자 언어에 근거하도록 만드는 핵심 데이터셋이 되었으며, 상세 내용은 5.1절에서 기술한다. 중간보고 시점의 핵심 변경은 다음 세 가지로 요약된다.

표 2.1: 시스템 설계 변경 사항

#	변경 축	착수보고서	중간~최종
1	AI의 역할	자기 칭찬 글 자동 재작성	짧은 코칭 제공 후 사용자가 직접 재작성
2	개인화 구조	감정 벡터, 시간 감쇠 리랭킹, 적응형 프롬프트	style_profile 기반 개인 기록 및 few-shot 예시 이중 검색
3	품질 보증	LLM CoT 기반 Self-Inspection	규칙 기반 검증 및 자동 재시도 루프

2.3 지도교수 피드백 및 반영 방향 - 최종 평가표 기준

표 2.2: 착수보고 피드백 및 반영 결과

피드백	반영 결과
장기기억은 복잡한 구조를 한 번에 적용하기보다 다양한 검색 방식을 실험하며 단계적으로 개선할 필요가 있음	감정 벡터 및 시간 감쇠 리랭킹 등 복합 모듈을 제거하고 pgvector 코사인 유사도 검색으로 단순화하였다. 데이터 축적 이후 리랭킹 도입 여부를 재검토한다.
피드에 부정적 글도 허용할 경우 서비스 정체성이 흐려질 수 있음	피드는 자기 칭찬/공정 기록 중심으로 유지하되, 작성 화면의 유도 문구로 긍정적 기록을 자연스럽게 촉진하는 방향으로 정리하였다.

2.4 기능적 요구사항 (최종)

착수보고서의 기능적 요구사항 중 F1, F3, F4를 수정하고 F8, F9를 신설하였다. 변경된 항목은 굵게 표시하였다.

표 2.3: 기능적 요구사항 (최종본)

ID	요구사항	설명
F1	회원가입 및 로그인	카카오·애플·구글 소셜 로그인을 제공한다.
F2	자기 칭찬 글 작성	사용자는 400자 이내의 짧은 글을 작성하고, 사진 첨부 및 공개 여부를 설정할 수 있어야 한다.
F3	AI 칭찬 코칭	AI는 일기 원문을 대신 재작성하지 않는다. 원문에 근거한 2~4문장의 짧은 코칭 한마디를 제공하고, 최종 게시글은 사용자가 직접 작성한다.
F4	위기 신호 및 유해 표현 감지	검사 대상은 (1) 자살·자해 암시 표현, (2) 앱 내부 규칙 위반 표현이며, 감지 시 생성을 차단하고 사전 정의된 안전 응답으로 대체한다.
F5	소셜 피드	사용자는 친구의 칭찬 글을 피드에서 확인하고, 댓글을 통해 응원할 수 있어야 한다.
F6	아카이브	사용자는 자신이 작성한 글을 캘린더 형식으로 확인하고, 작성 스트릭 및 기록 통계를 볼 수 있어야 한다.
F7	푸시 알림	연속 출석 시 보상 (캐릭터)을 제공하고, 친구가 글을 작성하거나 내 글에 반응하면 알림을 제공해야 한다.
F8	AI 응답 스타일 일관성 보장	생성된 한마디는 말투 규칙, 문장 수, 금칙 표현 검증을 통과한 경우에만 사용자에게 노출한다. 실패 시 교정 지시문을 부가하여 자동 재생성한다.
F9	개인 기록 기반 맥락 반영	사용자의 과거 기록 중 오늘의 일기와 의미적으로 유사한 항목을 검색하여 프롬프트에 주입한다.

2.5 비기능적 요구사항 (최종)

표 2.4: 비기능적 요구사항 (최종본)

ID	요구사항	설명
NF1	크로스플랫폼	React Native 기반으로 iOS 및 Android 환경에서 동일한 핵심 기능을 사용할 수 있어야 한다.
NF2	응답 속도	목표를 5초에서 3초 이내로 강화한다. 검증 실패 시 재시도가 발생하므로 재시도를 포함한 P95 기준으로 측정하며, Supabase 조회 2회를 병렬화하여 약 100ms를 절감한다.
NF3	데이터 보호	일기 및 AI 응답 데이터는 민감정보로 취급하며, AI 처리 전 PII를 마스킹한다. DB에 저장되는 개인 기록의 텍스트 필드는 AES-GCM으로 암호화하고, 원문 저장에 대한 별도 고지 및 동의 절차를 적용한다.
NF4	안전성	위기 대응 모드(전문기관 연계 등 개입)는 확장 기능으로 이관하되, 유해·위기 표현의 감지 및 차단은 MVP에 포함한다.
NF5	사용성	긴 작성 과정을 요구하지 않고, 모바일 환경에서 짧은 시간 안에 기록·재구성·게시가 가능해야 한다.
NF6	확장성	향후 AI 분석 리포트, 칭찬 챌린지, 장기 패턴 분석 등 확장 가능한 구조로 설계해야 한다.
NF7	재생성 제한	‘다른 한마디’ 요청은 최대 3회로 제한하여 API 비용을 감소한다.

제3장 시스템 설계 및 아키텍처

3.1 전체 시스템 아키텍처

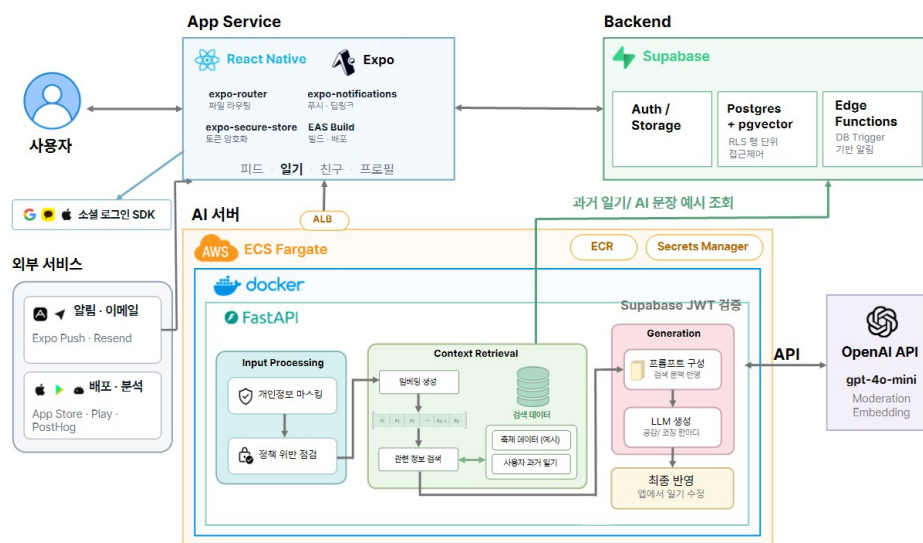


그림 3.1: 칭찬고래 시스템 아키텍처 개요

전체 시스템은 React Native/Expo 기반 모바일 애플리케이션, Supabase 기반 서비스 백엔드, FastAPI 기반 AI 서버의 세 계층으로 구성하였다. 모바일 애플리케이션은 Supabase Auth를 통한 사용자 인증과 PostgreSQL 기반 게시물·댓글·친구·알림 데이터 관리, Storage 기반 이미지 저장 기능을 사용한다. 신고, 계정 삭제 및 푸시 알림과 같이 서버 측 처리가 필요한 기능은 Supabase Edge Function과 데이터베이스 트리거를 활용하였다.

AI 기능은 별도의 FastAPI 서버로 분리하였다. 모바일 애플리케이션에서 인증된 사용자의 요청이 Cloudflare HTTPS 엔드포인트를 통해 AWS ECS에서 실행 중인 AI 서버로 전달되며, AI 서버는 OpenAI API와 Supabase pgvector를 이용하여 개인 기록 검색 및 코칭 응답 생성을 수행한다. 이를 통해 일반 서비스 기능과 AI 추론 영역을 분리하여 각각 독립적으로 개발·배포할 수 있도록 구성하였다.

구성요소 간 호출 흐름 전체 요청 흐름은 다음과 같다.

1. 모바일 클라이언트가 사용자의 일기 원문과 `retry_count`, 필요 시 캐시된 `style_profile` 을 담아 AI 서버(FastAPI)의 코칭 엔드포인트를 호출한다.
2. PII 마스킹으로 개인정보를 비식별화한 원문을 `text-embedding-3-small`로 1536 차원 벡터화하고, Supabase(pgvector)에서 유사 **few-shot 예시 3개**와 유사도 **0.35 이상의 개인 기록**을 조회한다.
3. 검색 결과·개인 기록·말투 선호·오늘의 일기를 하나의 프롬프트로 조합하여 `gpt-4o-mini` 를 호출한다.
4. 생성된 한마디는 규칙 기반 출력 검증기를 통과한 경우에만 클라이언트로 반환된다.
5. 사용자가 글을 직접 다시 쓴 뒤 ‘적용하기’를 누르는 시점에만 원문과 임베딩이 DB에 저장된다.

실패 시 동작 각 단계는 다음과 같은 실패 대응을 갖는다. Supabase 조회가 지연·실패하면 개인 기록/예시를 비운 채로 생성을 진행하여 응답 자체가 막히지 않도록 한다(개인 기록이 없는 신규 사용자와 동일 경로).

OpenAI 호출 실패 또는 타임아웃 시에는 재시도 예산 내에서 재호출하되, 최종 실패 시 사전에 정의한 **기본 응원 폴백 문구**를 반환한다.

출력 검증 실패 시 교정 지시문을 부가하여 자동 재생성을 수행하되, 실시간 응답 지연을 제한하기 위해 자동 보정은 최대 1회로 제한한다.

이 폴백/재시도 정책은 산학 자문 의견(6장)을 반영하여 명시적으로 정의되었다.

3.2 기술 스택

표 3.1: 주요 기술 스택

구분	기술	활용
모바일	React Native, Expo, Expo Router	iOS · Android 모바일 애플리케이션 구현 및 화면 구조 관리
서비스 백엔드	Supabase PostgreSQL, Auth, RLS, Storage, Edge Functions	사용자 · 게시글 · 댓글 · 친구 · 알림 데이터 관리, 인증 및 접근 제어, 이미지 저장
AI 서버	FastAPI, Python 3.11	AI 코칭 요청 처리 및 외부 AI 서비스 연동
AI	OpenAI API, pgvector	AI 응답 생성, 임베딩 생성 및 개인 기록 · few-shot 예시 검색
인프라	Docker, Amazon ECR, AWS ECS Express	AI 서버 컨테이너 이미지 관리 및 운영 배포
CI/CD	GitHub Actions, AWS OIDC	Docker 이미지 빌드, ECR 업로드 및 ECS 자동 배포
네트워크	Cloudflare	API 도메인 연결 및 HTTPS 적용
모니터링	CloudWatch, Server-Timing	운영 로그 수집 및 요청 단계별 응답 지연 분석
앱 배포	EAS Build, TestFlight	iOS 애플리케이션 빌드 및 실기기 테스트 배포

서비스 데이터와 인증은 Supabase를 활용하여 별도의 범용 백엔드 서버를 운영하지 않고 구성하였으며, AI 연산은 FastAPI 서버로 분리하였다.

AI 서버는 Docker 이미지로 표준화하여 Amazon ECR에 저장하고 AWS ECS Express 에서 실행하였다. GitHub Actions와 AWS OIDC를 연동하여 main 브랜치 변경 시 이미지 빌드부터 ECS 배포까지 자동화하였으며, 운영 환경의 API 키와 인증 정보는 AWS Secrets Manager를 통해 런타임에 주입하였다.

모델은 OpenAI API(gpt-4o-mini 생성, text-embedding-3-small 임베딩)를 사용한다. 데이터 계층은 Supabase(Postgres + pgvector)를 사용한다. 서빙(whale-AI)과 학습 파이프라인(whale-model)의 저장소를 분리하고, 통합 지점은 Supabase의 training_examples 테이블로 한정하였다.

제4장 구현 상세

4.1 AI 칭찬 코칭 파이프라인

시스템 프롬프트는 고래 캐릭터의 역할(사용자가 스스로 다시 쓰도록 돕는 짧은 한마디, 최종 게시글은 대신 완성하지 않음)과 작성 규칙(2~4문장, 감정 우선 인정, 원문에 없는 성취 날조 금지, 훈계·과장 금지)을 명시한다. 프롬프트는 v1에서 v4로 반복 개선되었으며, 주요 변경은 (1) 금지 기반 지시를 허용 어미 whitelist 기반으로 전환, (2) 다중 턴 few-shot 주입, (3) 위로+구체적 칭찬+마무리의 3단 구조 예시 재작성이다.

4.2 개인 기록 검색

일기 저장 시 원문을 임베딩하여 `user_memories`에 저장하고, 코칭 요청 시 오늘 원문을 임베딩하여 코사인 유사도로 과거 기록을 검색한다.

유사도 임계값 0.35 설정 근거. 임계값이 너무 높으면 표현은 다르지만 맥락상 관련 있는 기록이 걸려져 개인화 효과가 약해지고, 너무 낮으면 무관한 기록이 섞여 노이즈와 토큰 비용을 늘린다.

`text-embedding-3-small` 임베딩 공간에서 코사인 유사도 0.3대는 완전히 같은 주제는 아니지만 의미적으로 인접한 구간에 해당하여, 개인 기록이 적은 초기 단계에 맞춰 **0.35를 초기값**으로 채택하였다. 이론적으로 유도한 값이 아닌 초기 휴리스틱으로, 기록이 축적되면 재조정할 예정이다.

관련 기록이 없을 경우 신규 사용자와 동일하게 개인 기록 없이 프롬프트를 구성한다 (콜드스타트 대응).

4.3 출력 검증 및 자동 재시도

검증기는 말투 규칙, 문장 수, 금칙 표현 및 구조(JSON, 앵커)를 판정한다. 검증 실패 시 교정 지시문을 추가하여 자동 재생성을 수행하되, 운영 성능 계측 결과 반복적인 재생성이

응답시간 증가의 주요 원인으로 확인되어 자동 보정은 최대 1회로 제한하였다. 이에 따라 한 요청에서 GPT 생성 호출이 최대 2회를 넘지 않도록 조정하였다. 한편 사용자가 직접 요청하는 ‘다른 한마디’ 재생성은 비용 관리를 위해 별도로 최대 3회로 제한한다.

4.4 안전 계층

전문기관 연계나 적극적 개입을 포함한 완전한 위기 대응 모드는 MVP 범위에서 제외하였으나, 서비스 이용 과정에서 발생할 수 있는 위험 표현을 처리하기 위한 최소 안전 계층은 구현하였다. 입력 단계에서는 개인정보 비식별화와 OpenAI Moderation API 기반 필터를 적용하되, 한국어의 암시적 표현을 보완하기 위해 한국어 보조 분류 계층을 추가하였다. 탐지 대상은 자해·자살 암시, 폭력·협박, 혐오, 성적 표현으로 정의하였으며, 해당 표현이 탐지되는 경우 일반 AI 생성을 수행하지 않고 사전 정의된 안전 응답 및 상담 리소스를 제공하도록 분기하였다. 또한 생성 결과에는 구조화 출력 및 규칙 검증을 적용하였다.

안전 기준은 기획·AI 담당자가 공동으로 정의하였고, 자체 평가 데이터 292건으로 검증한 결과 유해 콘텐츠 탐지율 **89.5%**를 확인하였다.

4.5 모바일 앱 및 서비스 백엔드 구현

모바일 애플리케이션은 React Native와 Expo Router를 기반으로 구현하였다. 프로젝트 초기 구조와 전역 색상·폰트 디자인 시스템을 구성하고, 하단 탭을 중심으로 작성, 홈, 프로필, 보관함, 친구 요청 등 주요 화면 흐름을 설계하였다. 게시글 그리드, 파도 애니메이션, 보관함, 친구 요청·수락 등의 기능을 구현하고 iOS와 Android에서 동일한 핵심 기능을 사용할 수 있도록 구성하였다.

글쓰기 화면은 본문 입력, 사진 선택·그리드 표시, 공개 범위 설정 및 AI 피드백 확인을 하나의 작성 흐름으로 구성하였다. AI API를 연동하여 작성 중 코칭 한마디를 요청하고 확인할 수 있도록 하였으며, 최대 6장의 이미지 선택과 미리보기 등 세부 동작을 기능명세에 정리하였다. 또한 Android 환경에서 발생한 오류를 수정하여 플랫폼별 사용 흐름을 보완하였다.

일반 서비스 기능의 데이터 계층에는 Supabase를 사용하였다. PostgreSQL을 기반으로 사용자, 게시글, 댓글, 친구, 알림 데이터를 관리하고, Auth와 RLS를 이용하여 사용자별 데이터 접근을 제어하였다. 프로필 및 게시글 이미지는 Storage에 저장하며, 계정 삭제·신고 등 서버 측 처리가 필요한 기능은 Edge Function을 통해 처리하도록 구성하였다.

Google·Kakao·Apple 소셜 로그인을 통합하고 Supabase 세션을 SecureStore에 저장하여 자동 로그인 흐름을 구현하였다. AI 서버 호출 시에는 Supabase Access Token을 전달하고

서버에서 JWT를 검증하여, 요청 본문의 사용자 식별자를 직접 신뢰하지 않도록 구성하였다. 서비스 운영 환경을 정비하기 위해 Supabase 인도 리전에서 서울 리전으로의 마이그레이션을 수행하였다. 또한 서비스 도메인 `whale-done.com`을 구매하고 GitHub Pages 기반 HTML 페이지를 게시하여 서비스 안내와 정책 문서를 제공하는 경로를 마련하였다.

앱 출시 준비 과정에서는 이용약관·개인정보처리방침 및 동의 문구를 조사·작성하고, 신고·차단·회원탈퇴와 연령 제한 정책을 기능명세에 반영하였다. 게시글은 삭제 즉시 화면에서 숨긴 뒤 7일간 보관하고, 회원탈퇴는 최종 확인 후 계정과 관련 데이터를 즉시 영구 삭제하도록 정책을 구분하였다. 탈퇴 시 삭제 범위에는 게시글·이미지·소셜 활동 데이터와 AI 원본, 학습 후보 데이터 및 학습용 복사본·연결 정보를 포함하며, 법령상 보관 의무가 있는 데이터는 별도로 분리하도록 명시하였다. 또한 AI 생성 문구 표시를 화면 요구사항에 포함하고, 만 14세 미만 가입 제한을 소셜 로그인 연계 과정에서 확인할 운영 요건으로 정리하였다.

좋아요, 댓글, 친구 요청 등의 이벤트 발생 시 데이터베이스 트리거와 Edge Function을 통해 Expo Push Service로 알림을 전달하는 비동기 푸시 알림 구조를 구성하였다. 사용자별 푸시 토큰과 알림 설정을 관리하며 로그아웃 또는 만료 시 토큰을 정리하도록 구현하였다.

민감 데이터 보호 AI 기능에서 저장되는 개인 기록은 사용자의 일기 원문 및 AI 응답을 포함하므로 애플리케이션 계층에서 추가적인 보호가 필요하다고 판단하였다. 이에 `user_memories`에 저장되는 `content`, `edited_text`, `whale_message`, `rejected_messages`에 AES-GCM 기반 암호화를 적용하였다.

암·복호화는 FastAPI 서버에서 수행하며, 데이터베이스에는 평문 대신 암호문을 저장하도록 구성하였다. AES-GCM을 사용하여 데이터 기밀성과 함께 암호문의 위·변조 여부를 검증할 수 있도록 하였다.

단, 사용자 기록 검색에 사용하는 임베딩 벡터는 pgvector의 코사인 유사도 연산이 데이터베이스 내부에서 수행되어야 하므로 동일한 방식으로 암호화하지 않았다. 따라서 “검색은 벡터 데이터로 수행하고, 사용자에게 표시되는 원문 데이터는 별도의 암호문으로 저장”하는 구조를 적용하였다.

암호화 키는 소스 코드나 데이터베이스에 포함하지 않고 AWS Secrets Manager를 통해 운영 컨테이너에 주입하였다.

4.6 인프라 및 배포

AI 서버는 `python:3.11-slim` 기반 Docker 이미지로 패키징하여 로컬과 운영 환경의 실행 의존성을 통일하였다. 생성된 이미지는 Amazon ECR에 저장하고 AWS ECS Express에서 실행하도록 구성하였다. OpenAI 및 Supabase 관련 비밀값은 소스 코드나 Docker 이미지에

포함하지 않고 AWS Secrets Manager를 통해 컨테이너 실행 시 주입하였다.

배포 과정은 GitHub Actions와 AWS OIDC를 이용하여 자동화하였다. main 브랜치에 코드가 반영되면 Docker 이미지 빌드, ECR 업로드, ECS 서비스 이미지 갱신이 순차적으로 수행된다. 단순히 배포 API 호출 성공 여부만 확인하지 않고, 신규 이미지가 실제 ECS 서비스의 활성 이미지로 교체될 때까지 확인하여 배포 완료 여부를 검증하도록 구성하였다.

외부 접근을 위해 Cloudflare를 이용하여 api.whale-done.com 도메인과 HTTPS를 적용하였으며, /health 엔드포인트와 CloudWatch 애플리케이션 로그를 통해 서버 상태와 요청 처리 상황을 확인할 수 있도록 하였다. 모바일 애플리케이션은 EAS Build와 TestFlight를 이용하여 iOS 실기기 테스트가 가능한 배포 환경을 구성하였다.

4.7 운영 성능 계측 및 개선

병목 가설 및 계측 초기 실기기 테스트에서 AI 요청의 전체 응답시간이 약 11~13초까지 증가하는 사례가 발생하였다. 원인 분석을 위해 (1) 출력 검증 실패 시 최대 5회의 GPT 호출이 순차적으로 발생하는 서버 내부 재시도 구조와, (2) Cloudflare Edge-Worker-ECS Express를 거치는 네트워크 경로를 주요 병목 후보로 설정하였다.

표 4.1: AI 응답 지연 원인 분석

후보	측정 결과	판단
클라이언트 처리	토큰 처리 약 20~40ms	주요 병목 아님
인증/JWT	인증 및 핸들러 외부 처리시간이 거의 0초	주요 병목 아님
네트워크	개선 후 측정에서 클라이언트 왕복시간과 서버 처리시간 차이의 중앙값 약 0.62초	주 병목 아님
Moderation、임베딩、벡터 검색	분리 계측 후 Moderation 약 1.18초, 임베딩、검색 약 0.30초	일부 지연은 있으나 핵심 병목 아님
GPT 생성 및 Guard	9.30초 요청 중 7.60초 사용	핵심 병목

원인 분석 단계별 계측 결과, 9.30초가 소요된 요청에서 generate_with_guard가 7.60초를 차지하여 전체 서버 처리시간의 약 81.7%가 생성 및 출력 검증 단계에서 발생하는 것을 확인하였다.

회차별 로그를 추가하여 재생성 원인을 추적한 결과, 허용되지 않은 말끝 표현과 특정 금지 표현이 반복적으로 검증에 실패하면서 GPT 재호출이 연속적으로 발생하고 있었다.

개선 후 성능은 운영 API를 대상으로 워밍업 요청 1건을 제외한 일반 일기 입력 24건을 사용하여 측정하였다. 동시성은 1, 요청 간격은 6초로 설정하였으며, 각 요청에 대해 서버 처리시간과 GPT 생성시간을 기록하였다.

표 4.2: Guard 재시도 제한 적용 전·후 비교

지표	개선 전	개선 후
최대 GPT 호출 횟수	5회	2회
자동 보정 허용 횟수	최대 4회	최대 1회
관측 서버 처리시간	최대 9.30초	로컬 재측정 약 4.3초
병목 사례의 GPT 생성시간	7.60초	운영 측정 p50 3.09초 / p95 3.89초
운영 서버 전체시간	별도 분포 측정 없음	p50 4.87초 / p95 5.68초
운영 평균 GPT 호출 수	별도 측정 없음	1.67회

개선 및 재측정 개선 전 수치는 병목 분석 과정에서 관측된 개별 요청이며, 개선 후 운영 수치는 24건의 요청에 대한 분포 측정값이므로 두 값을 직접적인 성능 개선율로 해석하지 않았다. 다만 최대 생성 호출을 5회에서 2회로 제한하면서 장시간 반복 재생성으로 인한 지연을 구조적으로 제한할 수 있었다.

다만 재시도 횟수를 제한한 이후에도 별도 11건의 표본에서 첫 생성만으로 검증을 통과한 요청은 3건이었으며, 8건에서는 추가 생성이 발생하였다. 즉 최대 지연은 감소하였으나 출력 검증의 최초 통과율 자체는 여전히 낮은 상태로 확인되었다.

후속 개선 방향으로 말투 규칙을 단순 금지 목록으로 구성하기보다 허용되는 표현의 대조 예시를 프롬프트와 few-shot에 반영하고, 전체 응답을 다시 생성하는 대신 위반 문장만 부분적으로 수정하는 방법을 검토하였다.

운영 한계 또한 반복 GPT 호출은 응답 지연뿐 아니라 API 토큰 사용량에도 영향을 주었다. 부하 테스트 과정에서 요청 빈도가 증가할 경우 OpenAI의 TPM 제한으로 HTTP 429 응답이 발생하는 사례를 확인하였다. 따라서 Guard 재호출 감소는 단일 요청의 응답시간 개선뿐 아니라 동시에 처리할 수 있는 사용자 수를 확보하기 위한 운영상의 과제로도 확인되었다.

제5장 데이터 수집 및 성능 평가

5.1 학내 축제 부스 운영 및 실사용자 데이터 확보

부산대학교 축제 기간(2026.05.26~2026.05.28) 홍보 부스를 운영하여 초기 사용자 확보와 동시에 실사용자 원본 데이터셋 300건을 확보하였다. 부스에서는 두 유형의 작성을 요청하였다.

- 자기 칭찬 단문 작성 — “오늘 나를 칭찬할 만한 일”을 짧게 작성 (195건)
- 타인의 자기 칭찬에 대한 응원 응답 — 앞사람의 글에 응원 한마디를 다는 릴레이 (105쌍)

표 5.1: 축제 부스 원본 데이터셋 구성

구분	수량	스키마
single	195건	self_praise (자기 칭찬 원문 단독)
pair	105건	self_praise → human_reply (원문 + 실제 사람 응원)
합계	300건	

pair 데이터는 실제 사람이 사람에게 건넨 응원 텍스트로, LLM 생성 응답과 달리 또래 지지(peer support)의 언어를 담고 있어 핵심 데이터셋이 되었다.

5.1.1 데이터 정제 및 분류

원본 검토 결과 실제 사람의 응원 응답 다수는 코칭 구조(구체적 칭찬 포인트 언급) 없이 짧은 감탄형 반응이었다. 이는 소셜 피드 댓글로는 자연스럽지만 고래 캐릭터의 코칭 동작과는 성격이 다르므로, 용도별로 분류하여 배치하였다.

표 5.2: 원본 데이터 정제·분류 결과

데이터셋	수량	용도
tone_reference	228개	실제 사용자 말투·어미 참조 풀. 합성 생성 시 매 회 6개를 말투 힌트로 주입.
coaching_fewshot_real	18개	코칭 구조를 갖춘 응원 응답만 선별. 실서비스 few-shot 풀에 항상 포함 .

5.1.2 합성 데이터 생성 파이프라인

few-shot 풀의 카테고리 커버리지(자기비난, 번아웃, 작은 성취, 긍정 증폭, 관계)를 확보하기 위해 부스에서 수집한 실제 말투를 생성 과정에 지속 주입하여, 합성 데이터가 LLM 특유의 문체로 답변하는 것을 방지하였다.

5.2 AI 응답 품질 평가 프레임워크

AI 응답 품질을 정량적으로 검증하기 위해 3계층 KPI 구조를 설계하였다. 이는 산학 자문의견의 성능평가 권고(운영·품질·서비스 지표)와 정합한다.

표 5.3: 3계층 AI 품질 평가 KPI

계층	지표
① 운영 성능	검증기 1회 통과율, 재시도 포함 P95 응답 시간, 평균 재시도 횟수, 일일 API 비용
② AI 응답 품질	말투 규칙 준수율(규칙 기반 자동), HEART 기반 5축 점수 (LLM-as-a-Judge + 팀 3인), 블라인드 선호율
③ 사용자 가치	SENSE-7 설문, ‘적용하기’ 클릭률, 재생성 요청 비율, D1/D7 재방문을

평가 도구 선정 이유. 일반적인 텍스트 생성 지표(BLEU, ROUGE 등)는 정답과의 표면적 일치를 측정하므로, 정답이 하나로 정해지지 않는 정서적 지지·코칭 응답의 품질을 평가

하기에 부적합하다. 이에 지지적 응답의 품질을 다축으로 평가하도록 설계된 **HEART** [1] 프레임워크를 채택하여, 고래 한마디가 단순히 문법적으로 옳은지가 아니라 공감·구체성 등 코칭 역할에 필요한 속성을 갖추었는지를 측정한다(②계층).

한편 HEART가 응답 텍스트 자체의 품질을 본다면, 사용자가 실제로 그 응답에서 지지받았다고 느꼈는지는 별도로 확인해야 한다. 이를 위해 사용자가 지각한 지지·관계적 경험을 측정하는 **SENSE-7** [2] 설문을 세션 내에서 병행하여, 텍스트 품질(HEART)과 사용자 체감(SENSE-7)을 분리 측정한다(③계층).

평가 방법론상의 유의점. LLM-as-a-Judge에는 동일 계열 모델의 자기 향상 편향(self-enhancement bias)이 존재하므로, gpt-4o-mini 출력을 GPT 계열로 평가하지 않고 타 계열 모델을 심사자로 사용하거나 사람과의 상관을 함께 보고한다.

HEART 점수 산정은 실시간이 아니라 데이터 수집 종료 후 배치로 수행하며, MVP 기간에는 SENSE-7만 실시간으로 운영한다.

회귀 평가셋. 프롬프트 수정 시 기존에 잘 되던 케이스가 깨지지 않았는지 확인하기 위해, 대표 입력과 기대 출력 특성으로 구성된 소규모 회귀 평가셋을 유지한다. 이를 통해 인앱 데이터 수집 없이도 오프라인에서 반복 튜닝이 가능하다.

5.2.1 이용 지표 수집

사용자 가치 지표를 실제 이용 과정에서 측정하기 위해 모바일 앱에 PostHog 이벤트 수집 코드를 연동하고, 온보딩 완료, 친구 요청·수락, 게시글 작성, 댓글·좋아요 및 AI 한마디 이용을 총 9종의 커스텀 이벤트로 정의하였다. 특히 AI 한마디는 요청·재생성·적용·미적용 이탈을 구분하여, 코칭을 확인한 사용자가 이를 활용하는 과정을 분석할 수 있도록 구성하였다.

표 5.4: PostHog 커스텀 이벤트 정의

이벤트명	발생 시점	수집 속성
onboarding_completed	온보딩 프로필 저장 성공	tone_preference
friend_request_sent	친구 요청 전송 성공	없음
friend_added	친구 요청 수락 성공	friend_count
post_created	새 게시글 등록 성공	used_ai, char_count, photo_count, is_public

이벤트명	발생 시점	수집 속성
reaction_sent	댓글·답글 등록 또는 좋아요 추가 성공	type, post_id
ai_coach_requested	AI 한마디 요청	char_count
ai_coach_retried	AI 한마디 재생성 요청	retry_count
ai_coach_applied	AI 제안 적용	retry_count, edited
ai_coach_abandoned	AI 제안 미적용 상태로 닫기·이탈	retry_count

reaction_sent는 type 속성으로 댓글과 좋아요를 구분한다. 커스텀 이벤트 속성에는 일기 본문·댓글 내용·AI 응답 원문을 직접 포함하지 않고 글자 수 등 행동 분석에 필요한 속성을 기록하였다. 사용자 식별 정보와 SDK 자동 수집 정보는 별도로 처리된다.

5.3 베타 테스트 설계

클로즈드 베타는 축제 부스 참여자 명단을 활용하며, 목표 사용자는 30명이며, 설문 기간(10일) 동안 두 가지 설문을 진행한다. 설문은

1. 세션 내 일일 설문(Form 1, SENSE-7 6문항)
2. 1회성 마무리 설문(Form 2, 관계적 지속성·만족도·지속 사용 의향)

으로 구성된다.

제6장 산학 자문 의견 반영

중간보고서에 대한 산학협력 멘토의 의견을 검토하여, 항목별 보완·대응 방안을 다음과 같이 반영하였다.

표 6.1: 산학 자문 의견 및 대응

영역	자문 의견	대응 방안
결과물 양식	아키텍처 그림에 대한 본문 설명이 간략함. 호출 흐름과 실패 시 동작을 함께 서술 권장	3.1절에 클라이언트→AI 서버→OpenAI/Supabase 호출 흐름과 단계별 실패 대응(폴백/재시도)을 단락으로 추가
과제 목표	“응원 수신 시 재방문을 상승” 가설 검증을 위해 출시 전 이벤트 단위 로깅 준비 필요. GA4/Amplitude 권장	PostHog 기반 9종의 사용자 행동 이벤트 수집을 앱에 연동하여 AI 이용 및 소셜 상호작용 계측을 구현하였다. D1/D7 리텐션 집계와 응원 수신 여부에 따른 비교는 베타 운영 데이터 확보 후 수행하도록 구분하였다.
설계 사양	검증 실패 반복이 P95 3초와 충돌. 재시도별 지연 분포 로깅 및 최종 실패 폴백 문구 정의 권장	자동 보정을 최대 1회로 제한하여 한 요청당 생성 호출을 최대 2회로 제한하고, 최종 실패 시 폴백 문구를 반환(4.3절)
설계 사양	임계값 0.35 · 예시 3개의 선정 근거 기록, 신규 사용자 빈 기록 프롬프트 별도 정의 권장	수치 선정 근거를 보고서에 포함하고, 빈 배열 반환을 설계된 정상 동작으로 명시. 개인 기록이 없을 때의 콜드스타트 프롬프트 경로를 정의

영역	자문 의견	대응 방안
설계 사양	프롬프트 수정 시 회귀 확인용 소규모 평가셋 구축 권장	대표 입력·기대 출력 특성 기반 회귀 평가셋을 평가 프레임워크에 편입(5.2 절)
설계 사양	원문·임베딩은 파생 데이터이므로 회원 탈퇴 시 함께 삭제되는 경로 확인 권장	원문 저장 동의 항목(NF3) 외에, 탈퇴 시 <code>original_text</code> 와 임베딩이 함께 삭제되는 경로를 점검 항목으로 편입
설계 사양	ECS 자동 정지 후 첫 요청 콜드스타트 지연 우려. 헬스체크 통과 후 트래픽 수신 구조 고려 권장	재기동 시 헬스체크 통과 후 트래픽을 받도록 배포 구성을 조정하고, 첫 요청 지연을 P95 측정에서 분리 관찰
성능 평가	운영·품질·서비스 지표 중심 평가 권장(블라인드 선호 평가 병행)	3계층 KPI를 채택(5.2절). 블라인드 선호율을 품질 지표에 포함하고, 클로즈드 베타 풀로 가설 검증을 조기 시작

자문 결론에 대한 반영. 남은 기간의 우선순위로 제시된 (1) 재시도 포함 지연·비용 관
리, (2) 콜드스타트 사용자 경험, (3) 가설 검증용 이벤트 측정 준비를 최종 개발 우선순위에
반영하였다.

제7장 개발 일정 및 구성원별 역할

7.1 최종 개발 일정

표 7.1: 개발 일정표

업무	5월	6월	7월	8월	9월	10월
스터디 및 착수보고서 작성	•					
축제 부스 운영 및 사용자 데이터 수집	•					
기획·설계 확정		•				
개발 환경 세팅 및 Supabase 연동		•				
핵심 기능 개발 (작성·등록, 피드, 댓글, 아카이브)		•	•			
Supabase 데이터·인증·Storage 구축		•	•			
소셜 로그인 및 푸시 알림 구현				•		
운영 성능 계측 및 latency 최적화			•	•		
EAS/TestFlight 및 앱 배포 준비			•	•		
AI 파이프라인 설계 및 프롬프트 v2 확정			•			
few-shot 예시 풀 구축 및 품질 확인			•			
FastAPI AI 서버 구현 및 ECS 배포			•	•		
MVP 완성 및 내부 테스트			•	•		
AI 응답 품질 정량 평가 및 튜닝			•	•	•	
클로즈드 베타 및 가설 검증 (D1/D7)				•	•	
앱스토어 제출				•		
위기 신호 감지 시스템 (확장 기능)					•	•
최종 보고서 및 발표 준비						•

7.2 구성원별 역할 및 수행 내용

표 7.2: 구성원별 역할 및 주요 수행 내용

이름	담당 및 수행 내용
서혜민 (202355544)	PM · 기획 · 프론트엔드 사용자 사전 설문 기획·수행 및 응답 분석, 기능명세서·정보구조도 작성 디자인-기획 회의 주도, 핵심 화면 우선순위 및 디자인 일정·피드백 조율 축제 부스 기획·운영 및 오프라인 사용자 리서치 주도 글쓰기 화면 UI·기능 구현, AI 피드백 화면 퍼블리싱·API 연동 및 Android 버그 수정 이용약관·개인정보처리방침·동의 문구 작성, 신고·차단·연령 제한 및 회원탈퇴 시 관련 데이터 즉시 영구 삭제 정책 수립 서비스 도메인 구매 및 GitHub Pages 기반 정책 안내 웹페이지 게시 AI 안전·개인정보·평가 회의 진행 및 위기 입력 대응 요구사항 공동 정의 검증 근거를 고려한 온보딩 칭찬 스타일 유형검사의 MVP 제외 결정 Supabase 인도→서울 리전 마이그레이션 및 PostHog 9종 사용자 행동 이벤트 수집 코드 연동
김태란 (202355529)	서비스 개발 · 인프라 - React Native/Expo 프로젝트 구조 및 전역 디자인 시스템 구축 - 피드·보관함·친구·소셜 로그인 등 모바일 앱 주요 기능 구현 - Supabase 기반 데이터·인증·RLS·Storage·Edge Function 구성 - 민감정보 컬럼(content, edited_text 등) AES-GCM 암호화 적용 - 푸시 알림 및 사용자 계정 관련 기능 구현 - Docker·ECR·ECS Express 기반 AI 서버 운영 환경 구축 - GitHub Actions·AWS OIDC 기반 CI/CD 및 Cloudflare HTTPS 구성 - Server-Timing·CloudWatch 기반 응답 지연 계측 및 병목 개선 - EAS Build·TestFlight 기반 iOS 빌드 및 배포
오지현 (202355640)	AI 파이프라인 · AI 서버 착수보고 7단계 파이프라인을 MVP용 5단계로 재설계 프롬프트 v1→v4 반복 개선(허용 어미 whitelist, 다중 턴 few-shot, 3단 구조 예시 재작성) 규칙 기반 출력 검증기 및 자동 재시도 루프, 임계값 기반 style_profile 지시문 변환 로직 축제 부스 데이터(300건) 정제·분류 및 합성 데이터 파이프라인 구축 개인 메모리 테이블 스키마 및 사용자별 벡터 검색 RPC, ‘적용하기’ 기반 원문 저장(write) 경로 FastAPI AI 서버 구현 및 안전 계층 구현, 3계층 KPI 평가 프레임워크 설계

제8장 결론 및 향후 계획

8.1 결론

본 과제는 착수보고 이후 실사용자 데이터 수집, 프로토타입 검증, 범위 축소라는 제품 개발 사이클을 충실히 수행하였다. 핵심 성과는 (1) AI의 역할을 “자동 재작성”에서 “짧은 코딩 후 사용자가 직접 재작성”으로 재정의하여 서비스 핵심 가치를 구조적으로 보존한 점, (2) 비결정적 LLM 자가검수를 규칙 기반 검증기와 제한된 재시도 구조로 대체하여 응답 품질을 검증하면서 반복 재생성에 따른 지연을 완화한 점, (3) 축제 부스 실사용자 데이터 300건을 프롬프트·예시·평가의 근거로 활용한 점, (4) 운영·품질·서비스 3계층 KPI와 회귀 평가셋을 갖춘 평가 체계를 설계한 점이다. 또한 (5) React Native/Expo 모바일 애플리케이션부터 Supabase 기반 서비스 백엔드, AWS ECS 기반 AI 서버 배포 및 CI/CD까지 실제 서비스 운영에 필요한 전체 시스템을 통합 구축하고, 운영 환경에서 성능을 계측·개선한 점이다.

8.2 향후 계획

- **개인화 고도화:** 데이터 축적 후 시간 가중치 리랭킹 도입을 A/B로 검증하고, 개인 사실 슬롯(user_facts)의 상한 관리 로직을 고도화한다.
- **모델 파인튜닝:** 동의 기반 학습 데이터(training_examples) 축적 후 Qwen3 LoRA 파인튜닝을 통해 자체 언어 모델을 개발한다. 서빙(whale-AI)과 학습(whale-model)은 분리 구조를 유지한다.
- **위기 신호 감지 확장:** 최소 안전 계층을 넘어 완전한 위기 대응 모드를 확장 기능으로 개발한다.
- **수익 모델:** 누적 기록을 분석하는 AI 회고 리포트를 핵심 사용자 대상 유료 기능으로 검토한다.
- **가설 검증:** 클로즈드 베타에서 친구 응원 수신 여부에 따른 D1/D7 재방문을 차이를 측정하여 핵심 가설을 검증한다.

참고문헌

- [1] L. Iyer, K. Aggarwal, et al., *HEART: A Unified Benchmark for Assessing Humans and LLMs in Emotional Support Dialogue*. arXiv preprint arXiv:2601.19922, 2026. <https://arxiv.org/abs/2601.19922>
- [2] J. Suh, L. Le, E. Shayegani, G. Ramos, J. Amores, D. C. Ong, M. Czerwinski, & J. Hernandez, *SENSE-7: Taxonomy and Dataset for Measuring User Perceptions of Empathy in Sustained Human-AI Conversations*. arXiv preprint arXiv:2509.16437, 2025. <https://arxiv.org/abs/2509.16437>